

Resolution of the N-Queens Problem using Grover's Algorithm

Ignacio Andres Schwindt (03229/0) · Friday, June 17, 2026

ABSTRACT

We present an implementation of Grover's quantum search algorithm applied to the combinatorial N-Queens problem. The encoding assigns $k = \lceil \log_2 N \rceil$ qubits per queen to represent its column, eliminating row conflicts by construction. The oracle detects column and diagonal conflicts through explicit pattern enumeration and flag qubits, applying a -1 phase to valid configurations via the *compute-phase-uncompute* scheme. Full correctness is verified for $N = 4$ (14 qubits, 8 iterations, $P(\text{success}) \approx 99.3\%$), and the oracle is validated in isolation for $N = 5$ (30 qubits), confirming that all 10 known solutions are correctly identified. Scalability limitations and potential optimizations using quantum adders are discussed.

KEYWORDS Quantum computing · Grover's algorithm · N-Queens · Quantum oracle · Qiskit · Unstructured search

1. INTRODUCTION

The N-Queens problem involves placing N queens on an $N \times N$ chessboard such that no two queens threaten each other: they cannot share the same row, column, or diagonal. Originally formulated by Max Bezzel in 1848 and extensively studied by Gauss, the problem has become a classic in combinatorial theory and computer science [4]. For $N = 4$, there are exactly 2 solutions; for $N = 5$, there are 10; for $N = 8$ (standard chessboard), there are 92; and the number grows rapidly. Classical brute-force search has a complexity of $\mathcal{O}(N!)$ when exploring column permutations, and although backtracking algorithms improve this in practice, the worst-case complexity remains exponential.

Grover's algorithm [1], published in 1996, offers a quadratic speedup for unstructured search problems: if there are M solutions in a space of size $\mathcal{N}$, it finds them in $\mathcal{O}(\sqrt{\mathcal{N}/M})$ oracle queries, compared to $\mathcal{O}(\mathcal{N}/M)$ in the classical case. This speedup is optimal: Bennett, Bernstein, Brassard, and Vazirani proved that no quantum algorithm can do better for unstructured search [5].

In this work, we present a Python implementation using Qiskit [3], simulated with Aer, which covers the quantum encoding of the chessboard, the design of the oracle, Grover's diffuser, and results for $N = 4$ (full execution) and $N = 5$ (oracle validation).

Additionally, we discuss the impact of mapping combinatorial constraints to reversible logic gates. Although the physical execution of algorithms with such depth is still limited by the noise inherent in NISQ (*Noisy Intermediate-Scale Quantum*) processors, this exploration establishes a useful case study on how to structure and validate complex decision oracles modularly.

2. GROVER'S ALGORITHM

2.1 Principle and General Flow

Grover's algorithm operates on n qubits with $\mathcal{N} = 2^n$ states. The oracle U_f applies $U_f |x\rangle = (-1)^{f(x)} |x\rangle$ (phase kickback to the solutions). The algorithm (Fig. 1) has three stages: (1) Initialization in uniform superposition $|\psi_0\rangle = H^{\otimes n} |0\rangle^{\otimes n}$. (2) Grover's iteration (repeated t^* times): oracle U_f + diffuser $D = 2|\psi_0\rangle\langle\psi_0| - I$. (3) Measurement in the computational basis.

Measurement collapses the state of the system, revealing the solution with high probability if the correct number of iterations was chosen. This probabilistic behavior is typical of quantum algorithms, where constructive interference amplifies the amplitude of states that meet the oracle's conditions.

Grover's Algorithm for N-Queens

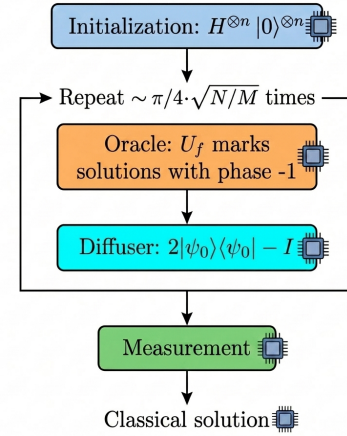


Figure 1. Flow of Grover's algorithm for N-Queens.

2.2 Geometric Interpretation

The state of the system lives in a two-dimensional plane spanned by:

$$|\alpha\rangle = \frac{1}{\sqrt{\mathcal{N} - M}} \sum_{f(x)=0} |x\rangle, \quad |\beta\rangle = \frac{1}{\sqrt{M}} \sum_{f(x)=1} |x\rangle \quad (1)$$

The initial state $|\psi_0\rangle$ forms an angle $\theta/2 = \arcsin \sqrt{M/\mathcal{N}}$ with $|\alpha\rangle$. Each iteration rotates the state by an angle θ towards $|\beta\rangle$.

2.3 Optimal Iterations

The optimal number of iterations is:

$$t^* = \left\lfloor \frac{\pi}{2\theta} - \frac{1}{2} \right\rfloor, \quad \theta = 2 \arcsin \sqrt{\frac{M}{\mathcal{N}}} \quad (2)$$

For $N = 4$: $\mathcal{N} = 256$, $M = 2$, $\theta \approx 0.177$ rad, $t^* = 8$, $P(\text{success}) \approx 99.3\%$. **Note:** If t^* is exceeded, the probability *decreases* — the algorithm "overshoots" the solution. This phenomenon has no classical analog.

3. QUANTUM ENCODING

3.1 Row Symmetry Elimination

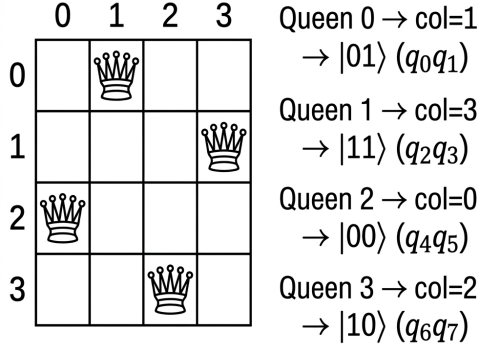
The first design decision is to implicitly fix each queen i in row i . This eliminates row conflicts by construction and reduces the search space from N^{2N} (arbitrary positions) to N^N (only columns per queen).

3.2 State Register

The column of queen i is encoded using $k = \lceil \log_2 N \rceil$ binary qubits. The qubits $q_{ik}, q_{ik+1}, \dots, q_{ik+k-1}$ represent the column in base 2,

with q_{ik} as the least significant bit (LSB). For $N = 4$: $k = 2$, total register of $n = Nk = 8$ qubits (Fig. 2).

Quantum encoding of N-Queens for N=4



Combined state: |01110010⟩

Figure 2. Encoding of [1, 3, 0, 2] for $N = 4$: 2 qubits per queen.

3.3 Ancilla Qubits (Flags)

The oracle requires auxiliary qubits: **Pair flags**: one qubit for each pair (i, j) with $i < j$, totaling $\binom{N}{2}$ flags. **Range flags** (only when $2^k > N$): one qubit per queen to detect columns outside $[0, N)$. For $N = 4$, $2^2 = 4 = N$, they are not needed; for $N = 5$ ($2^3 = 8 > 5$), 5 flags are added.

Table 1. Circuit resources according to N .

N	k	State	Pair flags	Range flags	Total	Sols.
4	2	8	6	0	14	2
5	3	15	10	5	30	10
6	3	18	15	6	39	4

4. ORACLE DESIGN

The oracle is the core component of the circuit. It applies a -1 phase exclusively to states representing valid configurations, in three stages (Fig. 3): *compute*, *phase flip*, and *uncompute*.

Structure of Grover's Oracle for N-Queens

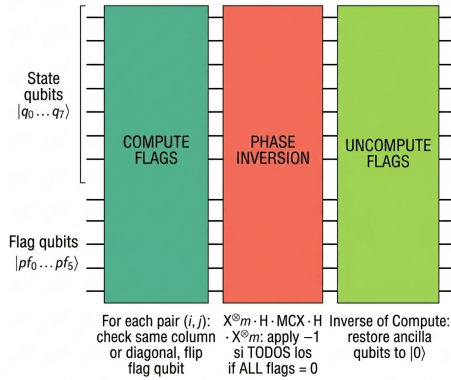


Figure 3. Structure: Compute → Phase Flip → Uncompute.

4.1 Compute Flags

For each pair (i, j) , $i < j$, a conflict ($c_i = c_j$ or $|c_i - c_j| = j - i$) is checked through **explicit pattern enumeration** (Fig. 4).

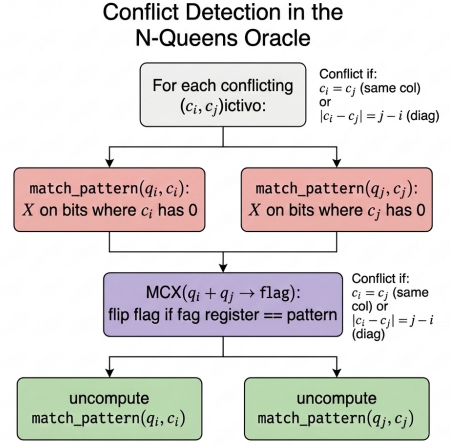


Figure 4. Conflict detection (c_i, c_j) with `match_pattern`.

4.1.1 Function `match_pattern`

This is the key piece for detection. Given a register of k qubits and a target value v , it applies X gates to each qubit whose corresponding bit in v is 0:

$$\bigotimes_{b=0}^{k-1} X^{1-v_b} |v\rangle = |1\rangle^{\otimes k} \quad (3)$$

This transforms the condition "the register equals v " into "all qubits equal 1", which is exactly what an MCX (generalized Toffoli) gate detects. A subsequent MCX flips the corresponding flag. Finally, the same sequence of X is applied to restore the register ($X^2 = I$). For a fixed computational state $|c_i, c_j\rangle$, exactly one pattern combination matches, guaranteeing that the flag is flipped 0 or 1 times.

4.1.2 Out-of-Range Detection

For N that is not a power of 2 (e.g., $N = 5$ with $k = 3$), column values $\{N, N+1, \dots, 2^k-1\}$ are invalid. An additional flag per queen activates if its column encodes a value $\geq N$, using the same `match_pattern` technique.

4.2 Phase Flip

Once all flags are computed, -1 is applied conditionally on all of them being 0 (no conflicts or out-of-range columns):

$$X^{\otimes m} \cdot (H \cdot MCX \cdot H) \cdot X^{\otimes m} \quad (4)$$

where m is the total number of flags. The $X^{\otimes m}$ sequence converts $|0\rangle^{\otimes m}$ into $|1\rangle^{\otimes m}$ (which triggers the MCX). The $H \cdot MCX \cdot H$ trick implements an MCZ (controlled phase):

$$HXH = Z \implies H \cdot MCX(\vec{c} \rightarrow t) \cdot H = MCZ(\vec{c}, t) \quad (5)$$

4.3 Uncompute Flags

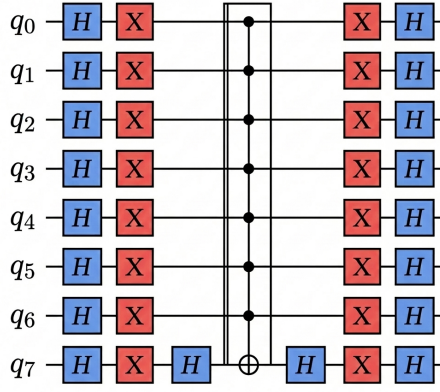
For the oracle to be unitary and reusable between iterations, ancillas must return to $|0\rangle$. The Compute stage is undone by applying the same operations in reverse order. Since X and MCX are self-inverses, this inversion is straightforward.

5. GROVER'S DIFFUSER

It implements the reflection $D = 2|\psi_0\rangle\langle\psi_0| - I$ (inversion about the mean) on the n state qubits. Its decomposition into gates is:

$$D = H^{\otimes n} \cdot X^{\otimes n} \cdot MCZ \cdot X^{\otimes n} \cdot H^{\otimes n} \quad (6)$$

The circuit (Fig. 5) operates by: (1) switching to the Hadamard basis with $H^{\otimes n}$; (2) marking $|0 \dots 0\rangle$ with $X^{\otimes n}$ followed by MCZ; (3) undoing $X^{\otimes n}$ and returning to the computational basis.



$$\text{Diffuser} = 2|\psi_0\rangle\langle\psi_0| - I$$

Figure 5. Diffuser circuit for 8 state qubits.

Effect on amplitudes. Let $\bar{a}$ be the mean amplitude. The diffuser transforms each amplitude $a_x \rightarrow 2\bar{a} - a_x$: states with amplitude below the mean are attenuated, while solutions (with large negative amplitude) are reflected to large positive values.

6. RESULTS

6.1 $N = 4$: Full Execution

For $N = 4$, the complete circuit was simulated with the AerSimulator backend (statevector method) for 4096 shots.

Table 2. Circuit parameters for $N = 4$.

Parameter	Value
State qubits	8 (2 bits/column)
Pair flags	6
Total qubits	14
Search space	$2^8 = 256$
Classical solutions	2
Optimal iterations	8
Depth (transpiled)	$\sim 41\,000$

The two valid solutions, $[1, 3, 0, 2]$ and $[2, 0, 3, 1]$, concentrate $\sim 99.3\%$ of the total probability ($\sim 49.6\%$ each, Fig. 6). Decoding the measured bitstring is done by reversing the bit order (Qiskit prints MSB first) and reconstructing each queen’s column: $\text{col}_i = \sum_{b=0}^{k-1} c_{ik+b} \cdot 2^b$.

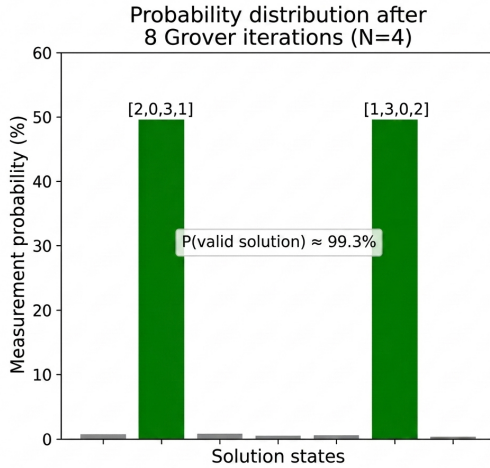


Figure 6. Probability after 8 iterations ($N=4$). The two solutions dominate with $\sim 49.6\%$ each; noise $< 1\%$.

6.2 $N = 5$: Oracle Validation

The complete circuit for $N = 5$ requires 30 qubits and ~ 44 iterations ($\sim 6 \times 10^5$ gates), which is beyond the practical reach of classical simulation. However, the oracle was validated in isolation

by preparing known computational states and measuring the flags:

The **10 classical solutions** (obtained via brute force with `classical_solutions`) resulted in 0 active flags in all cases. Tested negative cases included:

- $[0, 0, 0, 0, 0]$: all in the same column.
- $[0, 1, 2, 3, 4]$: main diagonal.
- $[0, 2, 4, 1, 7]$: column 7 out of range (≥ 5).
- $[1, 3, 5, 0, 2]$: column 5 out of range.

All were correctly rejected with flags > 0 . This confirms that the oracle perfectly separates valid configurations from the rest of the search space.

7. COMPLEXITY AND LIMITATIONS

7.1 Oracle Complexity

Explicit pattern enumeration yields: $\binom{N}{2} = \mathcal{O}(N^2)$ pairs of queens; up to $\mathcal{O}(N)$ conflicting combinations per pair; each check: $\mathcal{O}(k)$ X gates plus an MCX with $2k$ controls. Total: $\mathcal{O}(N^3)$ MCX calls and $\mathcal{O}(N^3 \log N)$ elementary gates. An implementation using **quantum adders** would allow for arithmetic column comparisons, reducing MCX gates per pair from $\mathcal{O}(N)$ to $\mathcal{O}(\text{poly}(k))$.

7.2 Scalability

Table 3. Simulation feasibility according to N .

N	Qubits	Iters.	Simulable
4	14	8	Yes (statevector)
5	30	44	Oracle only (MP5)
6	39	~ 600	No (classical simulator)

For $N \geq 5$, the two paths to end-to-end execution are: (1) real quantum hardware with sufficient fidelity and T_1/T_2 (aspirational in 2026); or (2) rewriting the oracle with quantum adders to reduce circuit depth.

8. CONCLUSIONS

1. **Verified Correctness.** For $N = 4$, the full algorithm finds the 2 solutions with $P \approx 99.3\%$. For $N = 5$, the oracle correctly classifies the 10 solutions and rejects all tested invalid cases.
2. **Quadratic Speedup.** The algorithm queries the oracle $\mathcal{O}(\sqrt{N/M})$ times, compared to $\mathcal{O}(N/M)$ in classical search. For $N = 4$: 8 iterations vs. ~ 128 classical queries.
3. **Modular Design.** The separation into encoding, oracle (compute-phase-uncompute), and diffuser allows for independent extension or replacement of components.
4. **Main Limitation.** Explicit enumeration generates deep circuits ($\sim 41\,000$ depth for $N = 4$). Quantum adders would significantly reduce the depth for $N \geq 5$.
5. **Perspective.** Execution on real quantum hardware would require coherence times (T_1, T_2) vastly superior to those currently available, along with quantum error correction. Nevertheless, this implementation serves as a conceptual demonstration of Grover’s quadratic advantage on combinatorial problems.

REFERENCES

- [1] L. K. Grover, “A fast quantum mechanical algorithm for database search,” *Proc. 28th ACM STOC*, pp. 212–219, 1996.
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge Univ. Press, 10th Ann. Ed., 2010.
- [3] Qiskit Development Team, “Qiskit: An open-source framework for quantum computing,” <https://qiskit.org>, 2024.
- [4] J. Bell and B. Stevens, “A survey of known results and research areas for n -queens,” *Discrete Math.*, vol. 309, no. 1, pp. 1–31, 2009.
- [5] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, “Strengths and weaknesses of quantum computing,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1510–1523, 1997.